EXPERIMENT-04 :

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets

CODE

```
# Artificial Neural Network using Backpropagation

# Dataset: XOR

# Manual implementation using NumPy


import numpy as np

import matplotlib.pyplot as plt


# 1. Training dataset (XOR)

X = np.array([

    [0, 0],

    [0, 1],

    [1, 0],

    [1, 1]

], dtype=float)


Y = np.array([

    [0],

    [1],

    [1],

    [0]

], dtype=float)
```

```
print("Training Dataset:")
print("Input  Output")
for i in range(len(X)):
    print(X[i], int(Y[i][0]))

# 2. Initialize weights and biases
np.random.seed(42)

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.zeros((1, output_neurons))

# 3. Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

# Derivative of sigmoid using its output
def sigmoid_derivative(output):
    return output * (1 - output)
```

```
# 4. Training using backpropagation
learning_rate = 1.0
epochs = 20000
loss_history = []

for epoch in range(epochs):

    # Forward propagation
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    # Binary cross-entropy loss
    eps = 1e-8
    loss = -np.mean(
        Y * np.log(predicted_output + eps)
        + (1 - Y) * np.log(1 - predicted_output + eps)
    )
    loss_history.append(loss)

    # Backward propagation
    output_error = predicted_output - Y
    delta_output = output_error
```

```
    hidden_error = np.dot(delta_output, W2.T)
    delta_hidden = hidden_error * sigmoid_derivative(hidden_output)

    # Update weights and biases
    W2 -= learning_rate * np.dot(hidden_output.T, delta_output) / len(X)
    b2 -= learning_rate * np.mean(delta_output, axis=0, keepdims=True)

    W1 -= learning_rate * np.dot(X.T, delta_hidden) / len(X)
    b1 -= learning_rate * np.mean(delta_hidden, axis=0, keepdims=True)

# 5. Test the trained network
hidden_output = sigmoid(np.dot(X, W1) + b1)
predicted_output = sigmoid(np.dot(hidden_output, W2) + b2)

predicted_classes = (predicted_output >= 0.5).astype(int)

print("\nActual Output:")
print(Y.astype(int).flatten())

print("\nPredicted Probabilities:")
print(np.round(predicted_output.flatten(), 4))

print("\nPredicted Classes:")
print(predicted_classes.flatten())

accuracy = np.mean(predicted_classes == Y) * 100
```

```
print("\nAccuracy:", accuracy, "%")

# 6. Test a new sample
new_sample = np.array([[0, 1]], dtype=float)

hidden_new = sigmoid(np.dot(new_sample, W1) + b1)
output_new = sigmoid(np.dot(hidden_new, W2) + b2)
class_new = (output_new >= 0.5).astype(int)

print("\nNew Input:", new_sample[0])
print("Predicted Output:", class_new[0][0])

# 7. Plot training loss
plt.figure(figsize=(8, 4))
plt.plot(loss_history)
plt.title("ANN Training Loss using Backpropagation")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.grid(True)
plt.show()

# 8. Plot actual vs predicted outputs
plt.figure(figsize=(8, 4))
positions = np.arange(len(X))

plt.plot(positions, Y.flatten(), "o-", label="Actual Output")
```

```
plt.plot(positions, predicted_classes.flatten(), "s--",
         label="Predicted Output")


plt.xlabel("Sample Number")
plt.ylabel("Output")
plt.title("Actual vs Predicted Output")
plt.xticks(positions, ["[0,0]", "[0,1]", "[1,0]", "[1,1]"])
plt.yticks([0, 1])
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

